



Programação Avançada 2025/26
04 Árvores | Implementação do ADT Tree

@Patricia Macedo & Bruno Silva

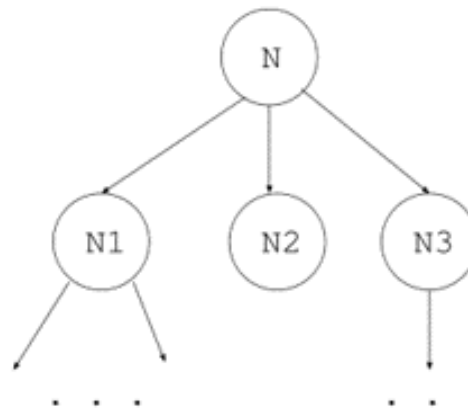
Trabalho Autónomo | Check Point (15 minutos)

Sumário

- ADT Tree
- Implementação
 - Conceito de Position
 - Interface Position
 - EAD recursiva para implementar a árvore
- Introdução ao exercício Prático

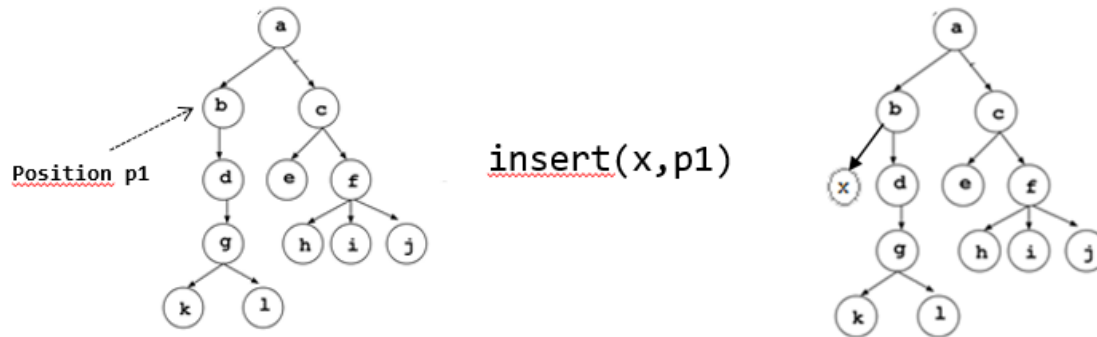
Tree - Características

- Uma árvore é composta por nós;
- No topo da árvore existe um nó especial, chamado raiz;
- Não possui ascendentes.
- Todos os outros nós têm exatamente um ascendente direto;



Noção de Posição

Se quisermos indicar que queremos colocar um novo elemento na árvore.
Como referenciar onde o colocamos?



- Para referenciar os nós da árvore sem expormos a sua implementação, introduzimos a noção de Posição (Position).
- A Posição modela a noção de “lugar” dentro de uma estrutura de dados, onde um único objeto é armazenado.
- É através da referência para o lugar na árvore que se insere e remove elementos numa árvore.

TAD Tree Operações Modificadoras

- `insert(x,pos)` : insere um elemento x como sendo filho do nó da árvore que está na posição pos , devolve erro se for uma posição que não pertence à árvore.
- `insert(x,n,pos)` : insere um elemento x como sendo o n -ésimo filho do nó da árvore que está na posição pos , devolve erro se for uma posição que não pertence à árvore, ou se n for um numero superior ao numero de filhos daquele nó.
- `remove(pos)` : remove o nó de uma determinada posição pos , devolve erro se for uma posição que não pertence à árvore.
- `replace (x, pos)` : substitui o elemento que se encontra na posição pos pelo valor x , devolve erro se for uma posição que não pertence à árvore.

TAD Tree Operações de Verificação

- `isInternal (pos)` : verifica se a posição `pos` se refere a um nó interno, devolve erro se for uma posição que não pertence à árvore.
- `isExternal (pos)`: verifica se a posição `pos` se refere a um nó externo, devolve erro se for uma posição que não pertence à árvore.
- `isRoot(pos)`: verifica se a posição `pos` se refere a um nó do tipo raiz, devolve erro se for uma posição que não pertence à árvore.
- `isEmpty`: verifica se a árvore está vazia.

TAD Tree Operações de Seleção

- size: devolve o tamanho da árvore.
elements: devolve uma coleção iterável com os elementos da árvore.
positions: devolve uma coleção iterável de posições da árvore.
- root: devolve a raiz da árvore
- parent (pos): devolve a posição do nó pai do nó que se encontra na posição pos, devolve erro se for uma posição que não pertence à árvore.
- children (pos) : devolve a coleção dos filhos de um nó da árvore que se encontra na posição pos, devolve erro se for uma - posição que não pertence à árvore.
remove(pos): remove o nó de uma determinada posição pos, devolve erro se for uma posição que não pertence à árvore.

Implementação em JAVA

Em JAVA para implementação de um ADT, iniciamos por especificar a interface.

```
public interface Tree<T>{  
    ...  
    //assinatura dos métodos  
}
```

Desafio: Faça a assinatura da seguinte operação em JAVA

- `insert(x,pos)`: insere um elemento `x` como sendo filho do nó da árvore que está na posição `pos`, devolve erro se for uma posição que não pertence à árvore.

Interface Position

Para referenciar os nós da árvore sem expormos a sua implementação, introduzimos a noção de Posição (Position).

```
/**
 * An interface for a position, which is a holder object storing a
 * single element.
 */

public interface Position<E> {
    /** Return the element stored at this position. */
    E element();
}
```

Interface Tree - método insert.

```
/**
 *
 * @param parent - Position of the ascendent node
 * @param elem - elem to insert in the tree
 * @return
 * @throws InvalidPositionException position of parent is null or is not referring to a treeNode
 */

public Position<E> insert(Position<E> parent, E elem) throws InvalidPositionException;
/**
```

TAD Tree: Estrutura de Dados

- Uma árvore é uma estrutura hierárquica que pode ser definida recursivamente:
 - Uma árvore é composta por **zero nós (vazia)** ou por **um nó (raiz)** que possui **um determinado número de filhos**, que por sua vez representam árvores menores (subárvores).
- Logo o EAD para implementar a árvore vai ter um cariz recursivo.
 - A recursividade é implementada através da definição dos **nós que compoem a árvore [TreeNode]**.

TAD Tree : TreeNode

O TreeNode implementa-se como uma InnerClass

```
private class TreeNode implements Position<E> {  
  
    private E element; // element stored at this node  
    private TreeNode parent; // adjacent node  
    private List<TreeNode> children; // children nodes  
  
    TreeNode(E element) {  
        this.element = element;  
        parent = null;  
        children = new ArrayList<>();  
    }  
  
    TreeNode(E element, TreeNode parent) {  
        this.element = element;  
        this.parent = parent;  
        this.children = new ArrayList<>();  
    }  
  
    public E element() {  
        if (element == null) throw new InvalidPositionException();  
        return element;  
    }  
}
```

TAD Tree : TreeLinked

A TreeLinked tem um único atributo que guarda o nó que é a raiz da árvore

```
public class TreeLinked<E> implements Tree<E> {  
  
    private TreeNode root;  
  
    public TreeLinked() {  
        this.root=null;  
    }  
  
    public TreeLinked(E root) {  
        this.root = new TreeNode(root);  
    }  
    ....  
}
```

Quiz Time | 30 segundos

1. :A interface Position

-  Serve para posicionar um nó no espaço 3D
-  Serve para referenciar os nós da árvore sem expormos a sua implementação
-  Serve para guardar os elementos da árvore
-  É uma implementação da inner classe TreeNode.

Quiz Time | 30 segundos

2. :Qual não é um atributo da InnerClass TreeNode ?



TreeNode root



List<TreeNode> children



E element



TreeNode parent

Quiz Time | 30 segundos

3. :Para instanciar uma Arvores de inteiros a instrução correta é:



```
TreeNode<Integer> root = new TreeNode();
```



```
Tree<Integer> tree = new TreeNode();
```



```
Tree<Integer> tree = new Tree();
```



```
Tree<Integer> tree = new TreeLinked();
```

Exercício

1. Faça clone do projeto base ADTTree_Template (projeto IntelliJ) do GitHub:

https://github.com/estsetubal-pa/ADTTree_Template

2. Observe o código da interface Tree, e da classe TreeLinked.

3. Observe o código da classe `TadTreeMain`

4. Complete o método

```
public E remove(Position<E> position) throws  
InvalidPositionException
```

Treino em casa

1. Altere o método main de forma a remover alguns nós da árvore e corra novamente o main.
2. Forneça o código dos métodos
 - `public boolean isExternal(Position<E> position) throws InvalidPositionException`
 - `public boolean isRoot(Position<E> position) throws InvalidPositionException`
3. Crie testes unitários para testar os dois métodos anteriores.

Nota: Poderá usar a estrutura da árvore fornecida no main, como base, para testar os métodos criados.

Soluções Quiz Time 🧐

1. :A interface Position

✌️ Serve para referenciar os nós da árvore sem expormos a sua implementação

2. :Qual não é um atributo da InnerClass TreeNode

✊️ `TreeNode root`

3. :Para instanciar uma Arvore de inteiros a instrução correta é:

👉 `Tree<Integer> tree = new TreeLinked();`